

Übungsblatt 2

Punkte: 17 / 20

Fortgeschrittene Algorithmen (Wintersemester 2025/26)

Bearbeitet von: Matthias Janßen, Raphael Thelen, Tillmann

Faust

October 28, 2025

Aufgabe 1

Sei im folgenden `roll( $\Omega, P$ )` eine Funktion, welche für einen Wahrscheinlichkeitsraum (Ω, P) Werte aus Ω anhand von P zieht.

a)

Sei $\Omega = \{K, Z\}$ und (Ω, P) unser Wahrscheinlichkeitsraum mit $P(K) = p, P(Z) = 1 - p$ mit $0 < p < 1$. Betrachte den Algorithmus:

```
1 GetFairCoinFromBiasedCoin( $\Omega, P$ )
2 {
3     do
4     {
5         v1 = roll( $\Omega, P$ );
6         v2 = roll( $\Omega, P$ );
7     } while (v1 == v2);
8     return v1 == 'K' ? 'K' : 'Z'; // returns 'K' iff v1 equals
9     'K' else 'Z'
10 }
```

Der Algorithmus wirft wiederholt zwei Münzen, bis beide Würfe ein unterschiedliches Ergebnis haben. Danach wird anhand daran, ob zuerst 'K' oder 'Z' geworfen wurde das Gesamtergebnis entschieden. Die Korrektheit des Algorithmus fußt darauf, dass KZ und ZK mit

$$P(KZ) = p(1 - p) = (1 - p)p = P(ZK)$$

dieselbe Wahrscheinlichkeit haben. Weiterhin sind einzelne Würfe unabhängig und die unerwünschten KK und ZZ können konsequenzlos verworfen werden.

✓ 2/2

b)

Sei analog nun $\Omega = \{1, 2, \dots, k\}$ und (Ω, P) der Wahrscheinlichkeitsraum mit $P(i) = p_i, i \in \Omega, 0 < p_i < 1$ und $\sum_{i \in \Omega} p_i = 1$. Die Methode `roll(P)` funktioniert analog zu 1a nur mit k möglichen Ergebnissen. Der Algorithmus

```

1 GetFairCoinFromBiasedDie( $\Omega, P$ )
2 {
3     do
4     {
5         v1 = roll( $\Omega, P$ );
6         v2 = roll( $\Omega, P$ );
7     } while (v1 == v2 or v1  $\notin$  {1,2} or v2  $\notin$  {1,2});
8     return v1 == '1' ? 'K' : 'Z'; // returns 'K' iff v1 equals
9         '1' else 'Z'
10 }

```

Ihr konstruiert hier einen fairen Münzwurf mit einem unausgewogenen k-seitigen Würfel, gefragt war aber tatsächlich ein fairer k-seitiger Würfel. Euer Ansatz geht aber in die richtige Richtung.

würfelt, analog zu 1a, so lange bis der Ergebniswurf je genau einmal '1' und '2' enthält. Anschließend wird anhand der Wurfreihenfolge das Ergebnis bestimmt. Die Korrektheit stützt sich weiter auf der Unabhängigkeit der Würfe und auf

$$P(12) = p_1 p_2 = p_2 p_1 = P(21)$$

Das praktische Problem, das sich ergeben kann ist, dass die Wahrscheinlichkeiten für '1' und '2' beliebig klein sein können und die Zahl der zu erwartenden Würfe bis zum Ergebnis damit beliebig groß.

1/3

Korrekt wäre: k mal würfeln, xtes Element aus einer Permutation mit k unterschiedlichen Elementen ist die gewürfelte Zahl. Sonst eben so lange neu starten, bis es mal hinhaut.

Aufgabe 2

Graph $G = (V, E)$ und drei Farben: Rot, Grün und Blau. Ziel: Färbung F , sodass die Anzahl der bichromatischen kanten maximal ist.

Eine randomisierte Strategie besteht darin, jedem Knoten zufällig eine der drei verfügbaren Farben zuzuweisen.

Algorithmus Random-3-color:

1. Initialisiere eine leere Farbzuzuweisung $F : V \rightarrow \{\text{Rot, Grün, Blau}\}$.
2. Für jeden Knoten $v \in V$:
 - Weise dem Knoten zufällig eine Farbe aus der Menge $\{\text{Rot, Grün, Blau}\}$ zu.

Sei $n = |E|$ nun die Gesamtzahl der Kanten im Graph, und sei X_i jeweils die Zufallsvariable zu einer Kante $e_i = (u, v) \in E$, sodass:

$$X_i = \begin{cases} 1 & \text{wenn } e_i \text{ bichromatisch} \\ 0 & \text{wenn } e_i \text{ monochromatisch} \end{cases}$$

Die Gesamtzahl der bichromatischen Kanten ist dann:

$$X = \sum_{i=1}^n X_i$$

mit Erwartungswert:

$$E[X] = \sum_{i=1}^n E[X_i]$$

Der Erwartungswert der Zufallsvariablen X_i ist gleich ihrer Wahrscheinlichkeit, also der Wahrscheinlichkeit, dass die jeweilige Kante e_i bichromatisch ist.

Das ist genau dann der Fall, wenn die Farbe von u ungleich der Farbe von v ist. Es gibt jeweils 3 mögliche Farben und bei zwei Knoten damit $3^2 = 9$ Farbkombinationen. Die Kante ist monochromatisch in drei von neun Fällen:

1. (Rot, Rot)
2. (Grün, Grün)
3. (Blau, Blau)

In den restlichen 6 Fällen ist sie bichromatisch. Die Wahrscheinlichkeit, dass eine e_i bichromatisch ist beträgt also:

$$P(X_i) = \frac{6}{9} = \frac{2}{3}$$

Der Erwartungswert entspricht wie zuvor festgestellt der Wahrscheinlichkeit, also:

$$E[X_i] = P(X_i) = \frac{2}{3}$$

und damit:

$$E[X] = \sum_{i=1}^n E[X_i] = n \cdot \frac{2}{3}$$

Die erwartete Anzahl an bichromatischen Kanten in einem mit diesem Algorithmus gefärbten Graphen beträgt dementsprechend $\frac{2}{3}$ der Gesamtzahl seiner Kanten.

Sehr ausführlich!

5/5

Aufgabe 3

a)

Der algorithmus terminiert garantiert da in jedem Schritt die Menge S um ein Element verkleinert wird bis

nur noch ein Element in S ist. Der Algorithmus ist korrekt. Aber warum?

1/2

b)

Jeder Rekursive Aufruf betrachtet eine Teilmenge $[S, S-1, S-2, \dots, 1]$. Im worst case werden alle

Elemente betrachtet also n Rekursive Aufrufe. In jedem Aufruf wird eine Konstante Zeit

benötigt.

D.h. $T(n) = T(n-1) + (n-1) \rightarrow T(n) = \mathcal{O}(n^2)$

2/2

c)

Nur wenn das kleinste Element ausgewählt wird, wird die Überprüfung in Schritt 5 des Algorithmus

durchgeführt. Dies passiert mit einer Wahrscheinlichkeit von $\frac{1}{n}$ im ersten Aufruf.

In diesem Fall liegt die Laufzeit bei $\mathcal{O}(n)$. In allen anderen Fällen wird

wird nur der Rekursive Aufruf durchgeführt. Dies passiert mit einer Wahrscheinlichkeit von

$\frac{n-1}{n}$.

Die erwartete Laufzeit $T(n)$ lässt sich also wie folgt beschreiben:

$$T(n) = T(n-1) + \frac{1}{n} \cdot \mathcal{O}(n)$$

$$T(n) = T(n-1) + \mathcal{O}(1)$$

$$T(n) = \mathcal{O}(n)$$

2/2

Aufgabe 4

Sei $\delta \in \mathbb{R}^+$ und Q ein $\delta \times \delta$ Quadrat. Bezeichne $P = \{p, q \mid p, q \in Q, \text{dist}(p, q) \geq \delta\}$ eine beliebige Teilmenge paarweise δ -distanter Punkte von Q

$$\mathbb{Z} : |P| \leq 4$$

Proof. o.B.d.A sei $Q = [0, \delta] \times [0, \delta]$.

1. $\mathbb{Z} : \exists P : |P| = 4$. Betrachte $P_1 = \{(0, 0), (0, \delta), (\delta, \delta), (\delta, 0)\}$. Offensichtlich gilt für alle Punkte $p, q \in P_1$: $\text{dist}(p, q) \geq \delta$ und $|P_1| = 4$. Weiterhin ist P_1 die einzige valide 4-elementige Punktmenge, die die Bedingung erfüllt, da sich keiner der Punkte in P_1 verschieben lässt ohne die Distanz zu den benachbarten Eckpunkten unter δ zu verringern.

2. $\mathbb{Z} : \forall P : |P| < 5$. Angenommen es gäbe ein P mit $|P| = 5$. Angenommen es gäbe ein P' mit $|P'| = 5$. Notwendigerweise muss $P_1 \subset P'$ gelten, da die Distanzeigenschaft sonst sofort verletzt wäre. Versucht man nun einen fünften Punkt p' hinzuzufügen kann dieser nicht platziert werden, ohne den Mindestabstand zu unterschreiten.

$$\Rightarrow \forall P = \{p, q \mid p, q \in Q, \text{dist}(p, q) \geq \delta\} : |P| \in \mathcal{O}(1)$$

□

4/4